

TXFETCH

Blockchain Transaction Retrieval Tool

Техническая документация

by xaynov (@ownstates)

1. Введение

TXFetch — утилита командной строки для поиска транзакций в блокчейне по косвенным признакам. Она решает обратную задачу: вместо того чтобы знать хеш транзакции и найти её данные, пользователь знает данные и хочет найти хеш.

Типичный сценарий использования: человек знает что отправил или получил перевод примерно в определённое время и на определённую сумму, но хеш транзакции утерян или неизвестен. TXFetch запрашивает данные у публичных API блокчейнов, фильтрует результаты и возвращает список кандидатов с оценкой вероятности совпадения.

1.1 Поддерживаемые сети

Сеть	Монеты	API	Фильтр времени	Пагинация
TRON	USDT (TRC-20)	TronGrid Events API	min/max_block_timestamp	fingerprint
TON	TON, USDT, NOT	TONAPI / TONCenter v3	start_date / end_date	before_lt / offset
ETH / EVM	USDT, USDC, ETH	Etherscan API	ts → block → tokentx	page / offset
Bitcoin	BTC	mempool.space	высота блока по времени	start_index (25 TX)

1.2 Входные данные

Обязательные параметры:

- Сеть (TRON, TON, ETH, BTC)
- Монета (USDT, TON, BTC и др.)
- Время транзакции в UTC

Необязательные параметры:

- Сумма с допуском $\pm 1\%$ по умолчанию
- Комментарий / мемо

2. Архитектура

Проект организован в виде единой точки входа и четырёх независимых адаптеров, каждый из которых реализует единый интерфейс.

```
txfetch/  
└─ main.py          # TUI, ввод параметров, вывод результатов
```

```
|— requirements.txt # зависимости
|— modules/
    |— models.py      # TXQuery, TXCandidate, score_candidate
    |— tron.py        # TRON / USDT-TRC20
    |— ton.py         # TON / Jetton
    |— eth.py         # EVM-сети
    |— btc.py         # Bitcoin
```

2.1 Поток данных

Пользователь вводит параметры → `TXQuery` → адаптер запрашивает API → локальная фильтрация → `score_candidate()` → список `TXCandidate` отсортированный по `confidence`.

2.2 Ключевые типы данных

TXQuery

Объект запроса, который `main.py` собирает из пользовательского ввода и передаёт в адаптер. Содержит временное окно (`time_from` / `time_to` в UTC), монету, опциональную сумму и мемо.

Важная деталь: если пользователь вводит время без секунд (например, 20:38), `time_from` устанавливается в 20:38:00, а `time_to` в 20:38:59. Это означает что поиск охватывает ровно одну минуту. Если секунды указаны явно — окно равно одной секунде.

TXCandidate

Унифицированный результат поиска. Все адаптеры возвращают список объектов этого типа, поэтому `main.py` не знает о внутренних различиях между сетями. Содержит хеш транзакции, время, сумму, адреса отправителя и получателя, мемо и значение `confidence`.

3. Адаптеры блокчейнов

Все адаптеры решают одну и ту же задачу разными методами: получить транзакции за нужный временной диапазон, поскольку ни один публичный API не поддерживает фильтрацию по сумме на стороне сервера.

3.1 TRON (tron.py)

Наиболее прямолинейная реализация. `TronGrid` поддерживает параметры `min_block_timestamp` и `max_block_timestamp` в запросе событий контракта, что позволяет получить только `Transfer`-события за нужный диапазон без дополнительных шагов.

Endpoint: `GET /v1/contracts/{contract}/events`

Параметры: `event_name=Transfer, min_block_timestamp, max_block_timestamp, limit=200`

Пагинация: `fingerprint` — курсор из поля `meta` ответа, передаётся в следующий запрос.

Особенности: TRON USDT использует 6 знаков после запятой (`decimals=6`), поэтому сумма хранится в единицах `sun` и делится на 10^6 . Стандартный `Transfer` в TRC-20 не содержит текстового комментария.

3.2 TON (ton.py)

Адаптер работает в двух режимах в зависимости от выбранной монеты.

Режим Jetton (USDT, NOT, USDC)

Endpoint: `GET /v2/jettons/{jetton_id}/transfers`

Параметры: `start_date, end_date (Unix seconds), limit, direction=in`

Пагинация: `before_lt` — logical time последней записи, уникальный механизм хронологии TON.

Комментарий к переводу (memo) в TON является нативным полем транзакции и хранится в `message_content.decoded.comment`. Это стандартная возможность блокчейна, не хак.

Режим нативного TON

Endpoint: `GET https://toncenter.com/api/v3/transactions`

Параметры: `start_utime, end_utime (Unix seconds), limit, offset`

TONAPI не предоставляет глобального endpoint с временным фильтром для нативных транзакций, поэтому для этого режима используется TONCenter v3.

3.3 EVM-сети (eth.py)

Ethereum и совместимые сети (BSC, Base) не поддерживают временные диапазоны в API напрямую — все запросы оперируют номерами блоков. Поэтому адаптер выполняет предварительный шаг конвертации времени в блоки.

Шаг 1 — конвертация: `GET`

`/api?module=block&action=getblocknobytime×tamp=...` — два вызова для `time_from (closest=after)` и `time_to (closest=before)`.

Шаг 2 — запрос: `GET`

`/api?module=account&action=tokenTx&contractaddress=...&startblock=X&endblock=Y` — без параметра `address`, только контракт токена.

Пагинация: `page / offset`, до 1000 записей на страницу.

Мультичейн поддерживается через словарь конфигураций: у каждой сети свой API URL и адреса контрактов. Добавление новой EVM-сети требует двух строк кода.

3.4 Bitcoin (btc.py)

Наиболее трудоёмкий адаптер. Bitcoin API не поддерживает ни временного, ни суммового фильтра — доступен только перебор блоков постранично.

Шаг 1 — оценка высоты: вычисляется из текущей высоты сети и разницы во времени по формуле `tip_height - (now - target_ts) / 600`. Bitcoin производит ~1 блок каждые 600 секунд.

Шаг 2 — поиск блоков: `GET /api/v1/blocks/{height}` возвращает 10 блоков за раз. Листаем пока блоки старше `time_from`.

Шаг 3 — транзакции блока: `GET /api/block/{hash}/txs/{start_index}` — 25 TX за вызов. Фильтрация по сумме полностью локальная.

Дополнительно: адаптер извлекает данные из `OP_RETURN` — специального поля куда некоторые сервисы записывают произвольный текст. Это аналог мемо в Bitcoin.

4. Алгоритм оценки (Scoring)

После получения транзакций от API каждому кандидату присваивается оценка `confidence` от 0 до 1. Кандидаты с оценкой ниже порога (по умолчанию 0.4) отбрасываются. Оставшиеся сортируются по убыванию.

Критерий	Вес	Логика
Время	0.5	Линейно от <code>time_from</code> . TX в 20:38:00 → 1.0, TX в 20:38:59 → ~0.0
Сумма	0.4	Точное совпадение → 1.0. В пределах <code>tolerance</code> → линейно. Вне — отброс.
Мемо	0.1	Вхождение строки (<code>case-insensitive</code>) → 1.0. Нет совпадения → 0.0.

4.1 Формула

```
confidence = 0.5 × time_score + 0.4 × amount_score + 0.1 × memo_score
```

4.2 Жёсткий отсев по сумме

Если пользователь указал сумму и отклонение кандидата превышает `tolerance` — функция немедленно возвращает 0.0, не вычисляя остальные компоненты. Это

исключает из выдачи транзакции с несовпадающей суммой независимо от их временного score.

4.3 Перераспределение весов

Если критерий не задан пользователем, его вес автоматически перераспределяется:

- Нет суммы и нет мемо → $\text{confidence} = \text{time_score}$
- Нет суммы, есть мемо → $0.9 \times \text{time} + 0.1 \times \text{мемо}$
- Есть сумма, нет мемо → веса 0.5 и 0.4 нормализуются пропорционально

5. Интерфейс командной строки

Утилита реализована как интерактивный TUI на библиотеке `prompt_toolkit`. Работает в два экрана.

Экран 1 — выбор сети

ASCII-арт с названием утилиты и подписью автора. Ниже — меню из четырёх сетей с навигацией стрелками $\uparrow \downarrow$ и подтверждением Enter.

Экран 2 — ввод параметров

Пошаговый ввод в четыре шага:

- Время транзакции в UTC (обязательно) — формат DD.MM.YYYY HH:MM или DD.MM.YYYY HH:MM:SS
- Монета (обязательно) — выбор из списка доступных для выбранной сети
- Сумма (необязательно) — Enter для пропуска
- Мемо / комментарий (необязательно) — Enter для пропуска

После ввода всех параметров выводится сводка и запрашивается подтверждение перед запуском поиска.

Вывод результатов

Результаты выводятся в виде таблицы: номер, полный хеш транзакции, confidence в процентах, сумма, время. При количестве кандидатов более 100 предлагается задать лимит или выгрузить все результаты в файл Excel.

6. Установка и запуск

6.1 Требования

- Python 3.10+

- `pip` или `pip3`

6.2 Установка

```
git clone <repo> && cd txfetch
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
```

6.3 Запуск

```
python3 main.py
```

6.4 Зависимости

- `requests` ≥ 2.31 — HTTP-запросы ко всем API
 - `prompt_toolkit` ≥ 3.0 — интерактивный TUI
 - `openpyxl` ≥ 3.1 — выгрузка результатов в Excel
-

7. Важные замечания

7.1 Время всегда в UTC

Все адаптеры работают исключительно с UTC. Пользователь обязан самостоятельно конвертировать локальное время в UTC перед вводом. Например, время 02:39 UTC+3 соответствует 23:39 UTC предыдущего дня.

7.2 Ограничения API

Публичные API без ключа имеют rate limit. TronGrid без ключа допускает ~1 запрос в секунду, Etherscan — 5 запросов в секунду. При превышении возвращается ошибка 429. Для стабильной работы рекомендуется получить бесплатный API-ключ на соответствующем сервисе.

7.3 Точность результатов

Утилита возвращает кандидатов, а не гарантированный ответ. При большом количестве переводов одинаковой суммы за одну минуту список может содержать несколько позиций с близким confidence. Для повышения точности рекомендуется указывать мемо если он известен.